23CSE455 – Design Patterns

# Design Pattern Analysis in a SvelteKit-Based Smart Home Application

Team A4

Case study: *ha-fusion*


| Roll number | Student name |
|---|---|
| AM.AI.U4AID23027 | Adwaith S Menon (AIDS) |
| AM.SC.U4CSE23153 | Sai Samay (AIDS) |
| AM.AI.U4AID23044 | Karthikeya Sai (AIDS) |
| AM.AI.U4AID23028 | Amitha K Anilkumar (AIDS) |
| AM.SC.U4CSE23052 | Meenakshi Manoj (AIDS) |


Submission repository: https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy

Report date: 11 October 2026

# Contents


**1 Application and architecture** **2**

1.1 Original source and environment . . . 2

1.2 Run and configuration commands . . . 2

1.3 Inspection scope . . . 2

1.4 Architecture and representative flows . . . 3

**2 Part 1 – Pattern discovery and verification** **3**

2.1 Method and scope limits . . . 3

2.2 Baseline totals and inventory . . . 4

**3 Part 2 – Evolution and refactoring** **5**

**4 Part 3 – Cross-language comparison** **5**

**5 LLM records and human verification** **6**

**6 Findings, contributions, and limitations** **6**

6.1 References . . . 7

**A Submission contents and checks** **7**

# 1 Application and architecture

## 1.1 Original source and environment

The case study examines *ha-fusion*, an open-source SvelteKit dashboard for Home Assistant: https://github.com/knowald/ha-fusion. The frozen baseline is commit `65cc1957c39c6005609102c33650c32abc8d12f0`; the recorded license is MIT. The latest stable release tag at the frozen default-branch HEAD is `2026.8.2`; its commit date is 2026-08-16. The inspection access date is 2026-10-11. The submission README records Node.js 22.15.0, pnpm 12.11.2, Svelte 5.56.8, SvelteKit 2.69.2, Vite 8.1.4, and Vitest 4.1.10. Static analysis used cloc 2.10 and Lizard 1.24.1. The application uses Home Assistant as its external entity and control service; the inspected project metadata does not identify a separate database for the in-scope flows.

The baseline application source is preserved in `data/code/`; the evolved source is in `Part2/code/`. Setup and test commands and inspected modules are documented in `README.md`. The baseline test log reports 13 test files and 46 tests passing. The final cumulative Part 2 logs report 15 test files and 68 tests passing, with `pnpm check` reporting zero errors and warnings. These are saved run results; see `data/test_result/` for the logs.

## 1.2 Run and configuration commands

Commands are run from the application root (`data/code/` for baseline, `Part2/code/` for evolved source):

| Command | Recorded status | Result / note |
|---|---|---|
| `pnpm install` | Run for baseline setup | Dependencies installed for the recorded baseline test run. |
| `pnpm dev - -open` | Not run in this review | Requires a configured, reachable Home Assistant instance. |
| `pnpm build` | Not run in this review | No build result is claimed. |
| `pnpm test` | Baseline and evolved logs | Baseline: 13 files / 46 tests pass. Evolved final: 15 files / 68 tests pass. |
| `pnpm lint` | Not run in this review | No lint result is claimed. |
| `pnpm check` | Evolved final log | Zero errors and zero warnings in the saved final check. |

The application README requires `HASS_URL`; `TZ` is optional. Development or end-to-end operation needs a running Home Assistant instance. The baseline `package.json` does not declare a Node engine constraint; the recorded environment uses Node.js 22.15.0 and pnpm 12.11.2.

## 1.3 Inspection scope

The pattern inventory covers the baseline collaboration records in `data/patterns.csv`. Inspected areas include dashboard/entity stores, recursive condition evaluation, Hearth configuration and history, entity modal selection, Home Assistant subscriptions, and the Picture Elements canvas. The full file/module inventory and exclusions (dependencies, generated output, static assets, and unrelated tooling) are in `README.md` and the Part 1 evidence. Communication details are discussed only where needed to explain the entity update flow; this report does not classify the protocol layer as a separate design pattern.

### 1.4 Architecture and representative flows

The UI uses Svelte components and stores for dashboard and Hearth state. Condition functions evaluate visibility trees; modal utilities select a domain-specific entity modal. The Socket connection receives entity updates from Home Assistant and publishes them through the `states` store for reactive UI consumers.

```
User click
  -> Main/Button.svelte click handler and toggle()
  -> Utils.getTogglableService() chooses domain.service
  -> home-assistant-js-websocket.callService(connection, ...)
  -> Home Assistant service / device
  -> Socket.ts subscribeEntities callback
  -> Stores.states
  -> Main/Button.svelte $states effect updates entity view
```

Representative source traces recorded in the inventory are:

- Entity state updates: `Socket.ts:107-116` subscribes to entity changes; `Stores.ts:23` exposes the state store; the entity UI consumes that store (`Main/Entities.svelte:1-50`).
- Visibility: `Types.ts:58-69` defines the recursive condition shape; `Conditional.ts:70-90,132-163` dispatches and recursively evaluates conditions.
- Entity modal selection: `Hearth/modals.ts:11-171` chooses a modal using the entity domain, with domain extraction in `Utils.ts:74-76` and modal invocation in `Modals.ts:17-21`.
- Canvas behavior: `konvaBase.ts:13-517` supplies shared canvas lifecycle behavior specialized by `konvaViewer.ts` and `konvaEditor.ts`.

The device-control path is traceable in the baseline source: the selected entity identifier is derived from `sel.entity_id` (`data/code/src/lib/Main/Button.svelte:41`); a non-edit-mode click invokes `toggle()` (same file, lines 755–767 and 189–218). That function asks `getTogglableService(entity)` to choose a domain-specific service (`data/code/src/lib/Utils.ts:96-149`) and sends the selected service with `entity_id` through `callService($connection, ...)` (`Main/Button.svelte:239-244`). The Socket connection subscribes to Home Assistant entity updates and writes them to `states` (`data/code/src/lib/Socket.ts:107-`
`Stores.ts:21-24`); the button effect reads `$states[entity_id]` to refresh its entity and optimistic UI state (`Main/Button.svelte:100-118`). This traces the application call to the Home Assistant library boundary; network delivery and the remote device response are outside this source-level trace.

Preferences/configuration have a distinct persistence flow. The page server loads `data/configuration.yaml` and `data/dashboard.yaml`, adds the Home Assistant URL from the environment or proxy request, and returns them to the UI (`data/code/src/routes/+page.server.ts:33-67`). Hearth edit state is mutated through `updateConfig()` and saved by `saveEdit()` as a revisioned POST (`data/code/src/lib/Hearth/store.ts:43-5`
the server serializes YAML and atomically writes the configuration file (`save_hearth/+server.ts:94-140`). These configuration values affect dashboard layout, widgets, theme, localization, and application settings. No separate database-backed preference store is identified in this trace.

## 2 Part 1 – Pattern discovery and verification

### 2.1 Method and scope limits

For Part 1, we used the frozen source to record participant roles, source locations, and distinct collaborations. The saved historical prompt, model output, and review are in `Part1/llm/`. We separated two independent Memento collaborations that the initial output had combined and added source locations to the inventory. A follow-up audit of the assigned smart-device, control, and preference scope is now recorded in `Part1/llm/p1_discovery_audit.md`; its confirmed-looking-only draft is `Part1/llm/patterns_p1_discovery.csv`. The follow-up is a source-based addition and is not presented as a historical model response.

That scoped audit records five confirmed-looking collaborations, two uncertain candidates, and five rejected candidates. It is limited to the inspected P1 paths listed in the audit note and makes no repository-wide exhaustiveness claim. The canonical `data/patterns.csv` and its totals remain the P2-owned baseline inventory; the P1 draft is a discovery handoff, not a replacement for those final records.

### 2.2 Baseline totals and inventory

The recorded counts are five pattern types and eight pattern instances: three Memento, two Composite, one Observer, one Factory Method, and one Template Method. The count file records two distinct concrete participant classes (`KonvaEditor` and `KonvaViewer`) under its stated counting rule. A consolidated unique-participant-symbol total is **UNVERIFIED**: the P2 inventory participant fields combine role aliases, variants, shorthand, and grouped products and have not been normalized into a symbol-level ledger. The scoped P1 draft separately deduplicates 32 participant symbols, as listed in its audit note. The P2-owned canonical inventory contains only its confirmed rows; the scoped P1 audit separately records two uncertain and five rejected candidates.

| Pattern | Instance | Principal source participants and evidence location |
|---|---|---|
| Memento | Classic Dashboard Edit History | `dashboard`, `history`, `HistoryButtons.svelte`; `data/code/src/lib/Stores.ts:29,147-152`; `Drawer/HistoryButtons.svelte:18,43-82`. |
| Memento | Hearth Config History & Edit Transaction | `hearthConfig`, snapshot stacks and edit operations; `data/code/src/lib/Hearth/store.ts:18,32-110`; configuration type in `Hearth/config.ts:324-341`. |
| Memento | Konva Editor History | `KonvaEditor` and `KonvaHistory`; `Modal/PictureElements/konvaEditor.ts:754-840`; type in `data/code/src/lib/Types.ts:350-353`. |
| Composite | Dashboard Section Layout Hierarchy | recursive `Section` and `findInSections`; `data/code/src/lib/Types.ts:42-56`; `data/code/src/lib/Utils.ts:25-60`. |
| Composite | Visibility Condition Tree | `Condition`, `validateCondition`, and `handleAnd`/`handleOr`; `data/code/src/lib/Types.ts:58-69`; `data/code/src/lib/Conditional.ts:58-216`. |
| Observer | Home Assistant WebSocket Entity Subscriptions | connection subscription, `states` store, and UI consumers; `data/code/src/lib/Socket.ts:107-116`; `data/code/src/lib/Stores.ts:23`; `data/code/src/lib/Main/Entities.svelte:1-50`. |
| Factory Method | Entity Domain Modal Factory | `openEntityModal`, `getDomain`, `openModal`, and concrete modal components; `data/code/src/lib/Hearth/modals.ts:11-171`; `Utils.ts:74-76`; `Modals.ts:17-21`. |
| Template Method | Picture Elements Konva Canvas Base | `KonvaBase`, `KonvaViewer`, and `KonvaEditor`; `konvaBase.ts:13-517`; `konvaViewer.ts` and `konvaEditor.ts` in the same directory. |

Full P2-owned role mappings are in `data/patterns.csv`; the scoped P1 discovery draft and disposition ledger are in `Part1/llm/`. Editable and rendered architecture diagrams are in `Part1/pattern_architecture/`. The source audit note records that the first model output combined distinct history mechanisms; the final inventory separates them. Strategy is not a baseline type in this inventory; it first appears during Change 1.

# 3 Part 2 – Evolution and refactoring

Three changes were implemented on the baseline and recorded in `data/patterns_evolution.csv`. Requirements, acceptance criteria, diffs, and logs are under `Part2/code/changes/Change1-Change3/`.

| Change | Requirement and design work | Verification and measured effect |
|---|---|---|
| CH01 | Add recursive `not` evaluation and dispatch through a function-based Strategy map. Extends the Visibility Condition Tree Composite and introduces the evaluator Strategy. | Final cumulative suite: 68 tests pass; type check: zero errors/warnings. Measured `Conditional.ts` + `Types.ts`: 468 to 484 LOC; `validateCondition()` CCN 6 to 3. |
| CH02 | Extract bounded Hearth undo/redo state into generic `HistoryManager<T>` using Extract Class and Encapsulate Collection. | Final cumulative suite: 68 tests pass; clean type check. Measured Hearth directory: 17,063 to 17,186 LOC; maximum history-operation CCN remains 2. |
| CH03 | Add configurable alert rail widget and dispatch configuration predicates through `WIDGET_CONFIG_VALIDATORS`. | Final cumulative suite: 68 tests pass; clean type check. Measured Hearth directory: 17,186 to 17,293 LOC; `railWidgetNeedsConfiguration()` CCN 7 to 2. |

Pattern effects, source-version totals, limitations, and exact logs are recorded in `Part2/Part2_Report.md`, `data/patterns_evolution.csv`, `data/pattern_counts.csv`, and `Part2/evolution_metrics.csv`. Across tracked snapshots, the inventory moves from 5 types/8 instances at the baseline to 6 types/10 instances after CH03. One CH03 Factory Method candidate was rejected because the registry and renderer branches do not show the Factory Method collaboration. Baseline pattern rows were carried forward in the evolution table; CH01–CH03 did not perform a new exhaustive Part 1 sweep.

| Snapshot | Types | Instances | Concrete classes | Rejected | Uncertain |
|---|---|---|---|---|---|
| Original | 5 | 8 | 2 | 0 | 0 |
| After CH01 | 6 | 9 | 2 | 0 | 0 |
| After CH02 | 6 | 9 | 3 | 0 | 0 |
| After CH03 | 6 | 10 | 3 | 1 | 0 |

These are tracked-inventory counts. Baseline rejected/uncertain zeros mean no such rows were carried into the inventory; they do not represent the result of an exhaustive baseline candidate sweep.

# 4 Part 3 – Cross-language comparison

Six selected collaborations were implemented as small behavior-matched examples in Java, Python, JavaScript, and C++: Composite / Visibility Condition Tree; Observer / Entity Subscriptions; Memento / Hearth History; Strategy / Visibility Condition Evaluator; Factory Method / Entity Modal; and Template Method / Canvas Renderer. The common fixtures and expected behavior are in `Part3/test-cases/common_spec.md`; production snippets and separate runners are grouped by pattern and language.

| Language | Mean LOC | Mean max CCN | Saved suites | Local rerun | Mean saved ms |
|---|---|---|---|---|---|
| Java | 68.0 | 5.0 | 6/6 pass | 6/6 pass | 8.196 |
| Python | 59.7 | 4.8 | 6/6 pass | 6/6 pass | 0.028 |
| JavaScript | 61.3 | 3.7 | 6/6 pass | 6/6 pass | 0.301 |
| C++ | 97.0 | 4.7 | 6/6 pass | not rerun here | 0.038 |

The canonical per-pattern/per-language data, exact runtimes and commands are in `data/cross_language.csv`; test logs and recomputed static metrics are in `Part3/evidence/`. Python, JavaScript, and Java runners passed locally (18 suites) on Python 3.12.2, Node.js 22.15.0, and OpenJDK 17.0.10. The six C++ runners were not rerun on this macOS host because their timing harnesses require Windows headers and performance-counter APIs; saved Windows logs report their earlier MinGW runs passing.

LOC excludes blanks and comments. Maximum function CCN is measured with Lizard 1.24.1; C++ counts include both implementation and header. The saved timing values are medians of five measured runner-body runs after one warm-up. They exclude process creation and VM/interpreter startup. The original timing hardware and operating system were not preserved, so these small test timings are not a controlled cross-language performance ranking. The earlier slide PDF is retained as historical output; its C++ LOC omitted headers.

Java uses declared interfaces/classes and runtime dispatch; Python supports abstract base classes and dynamic/structural use; JavaScript class syntax uses prototype-based objects and dynamic dispatch; C++ examples use abstract base classes and virtual functions. These are language-specific expression choices, not evidence that one language is superior. References to the relevant language documentation are listed at the end.

## 5 LLM records and human verification

AI-assisted work and its verification are recorded in `data/llm.csv`; supporting prompts, inputs, and review notes are stored with each part. The Part 1 record describes how we separated the merged Memento collaborations and added source locations. The Part 2 completion record notes that the historical CH01–CH03 model prompts and outputs were unavailable. The current reference task specification is labeled as reconstructed, not historical. For Part 3, the earlier raw model response was unavailable. Its prompt/context and the later completion review are preserved, and the later review is explicitly not presented as the earlier raw response. Evidence and limitations are in `Part2/llm/verification.md` and `Part3/evidence/verification.md`.

The current Part 1 task specification is stored in `Part1/Part1_prompt.txt`. The shorter historical Gemini prompt is preserved separately at `Part1/llm/historical_discovery_prompt.txt` and is the prompt linked to the saved historical output. Part 2's reference prompt is explicitly marked as reconstructed; it is not presented as a recovered historical transcript.

## 6 Findings, contributions, and limitations

The baseline inventory describes undo/redo in three separate subsystems, recursive compositions for layouts and conditions, a domain-specific modal selection collaboration, an entity-state subscription flow, and a shared canvas lifecycle specialized by viewer/editor subclasses. The evolution added condition negation, extracted history management, and introduced an alert widget; test evidence records behavior preservation and new behavior. The cross-language examples retain the same stated fixtures, but their small test workloads and differing historical runtime environments limit conclusions about performance.

| Team member | Contribution recorded from the team's work-division note |
|---|---|
| Adwaith S Menon | Part 1 pattern discovery. |
| Sai Samay | Part 1 pattern verification/counts . |
| Karthikeya Sai | Part 1 pattern /inventory; Part 2 evolution and refactoring. |
| Amitha K Anilkumar | Part 3 implementations in Java, Python, and JavaScript. |
| Meenakshi Manoj | Part 3 C++ implementation and report writing. |

The team confirmed that Amitha K Anilkumar implemented the Part 3 examples in Java, Python, and JavaScript, while Meenakshi Manoj handled the report and C++ implementation. Karthikeya Sai handled Part 1 verification and Part 2 evolution/refactoring, and all five members are in the AIDS batch. The other contribution descriptions follow the work- division details supplied by the team.

### 6.1 References


- Ha-Fusion repository and baseline: https://github.com/knowald/ha-fusion; MIT License.
- Oracle Java Tutorials, Polymorphism: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html.
- Oracle Java Tutorials, Defining an Interface: https://docs.oracle.com/javase/tutorial/java/IandI/interfaceDef.html.
- Python 3.12 documentation, Abstract Base Classes: https://docs.python.org/3.12/library/abc.html.
- MDN Web Docs, JavaScript Classes: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes.
- C++ working draft, virtual functions: https://eel.is/c++draft/class.virtual.
- Gamma, Helm, Johnson, and Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- Larman, *Applying UML and Patterns*, 3rd ed., Prentice Hall, 2004.
- cloc 2.10 and Lizard 1.24.1 measurement tools; versions and rules are documented in `README.md` and `Part3/evidence/measurement_notes.md`.


## A Submission contents and checks

This team folder contains the consolidated report source and compiled PDF, README, the canonical datasets in `data/`, original source and test logs, Part 1 diagrams and evidence, Part 2 changes and snapshots, and Part 3 snippets, shared test cases, evidence, and report. The course instructions request one `A4/` team folder and one pull request to the upstream repository's main branch. Capture the requested repository screenshot after the team folder is uploaded. Before submission, make sure the PDF is compiled from the final version of this TeX source.